Click here to view the PDF version.

```julia
begin
    using Plots
    using PlutoUI
    using PlutoTeachingTools
    using LaTeXStrings
    using LinearAlgebra
    using HypertextLiteral: @htl, @htl_str
end
```

Table of Contents

Boundary value problems

Stationary heat equation in one dimension

Differentiation matrices

Collocation method

Numerical stability

⚠ TODO ⚠

Galerkin methods

Discretisation of the weak form

Example: Sine basis

Comparison and conclusion

⚠ TODO ⚠

⚠ TODO ⚠

Optional: Finite elements

⚠ TODO ⚠

Boundary value problems

The goal of boundary value problems is to reconstruct the function values $u(x,t)$ of a function u on an entire domain $x \in [a,b]$ and for all times t knowing only two things

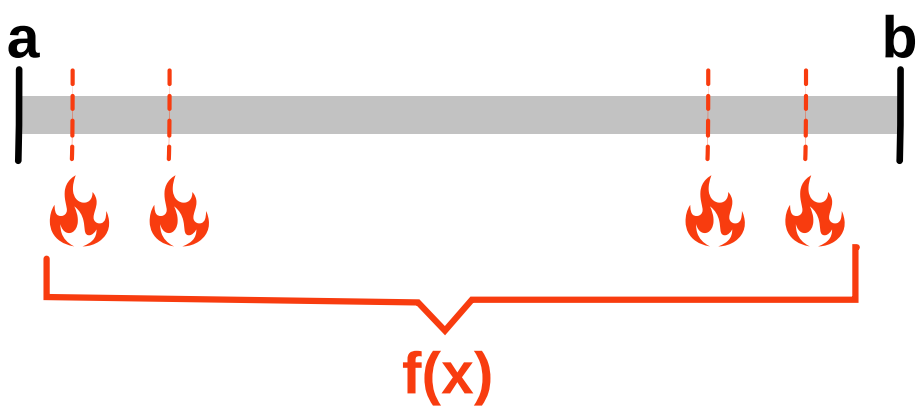
- Some information about u at the boundary $x = a$ and $x = b$ for the initial time $t = 0$. For example we may know the values $u(a,0)$ and $u(b,0)$.
- The relationship between u and its derivative at each point in space-time (x,t).

Classic examples of boundary value problems include

- the heat equation, where $u(x,t)$ is the temperature at time t and position x of a material — and the flux of heat at (x,t) itself depends on the temperature at this point in space and time, or
- Poisson's equation where $u(x,t)$ is the electrostatic potential at position x and time t corresponding to a given time-dependent density of electronic charges — where the charge itself determines the electrostatic potential acting at each point (x,t), or
- the diffusion equation where $u(x,t)$ is the concentration of some solute at time t at position x in the solution.

We will first consider the heat equation as one example.

Stationary heat equation in one dimension



We consider a metal bar extending over the interval $[a, b]$. For a given moment in time t we denote the **temperature** at a point $x \in [a, b]$ by $u(x, t)$. The **heat flux** we fruther denote by $\phi(x, t)$ and allow for an **external source of heat** $f(x)$, e.g. by one more more heat guns pointed at the metal bar.

The **rate of change of the internal heat** $\frac{\partial Q}{\partial t}$ at point x of the rod is proportional to the change in temperature $\frac{\partial u}{\partial t}$, that is

$$\frac{\partial Q}{\partial t} = c\rho \, \frac{\partial u}{\partial t}(x, t)$$

where proportionality constants are the heat capacity c and the mass density ρ.

Due to **conservation of energy** the change of internal heat of a slice $[x, x + \delta x]$ is equal to the incoming heat $f(x)\,\delta x + \phi(x, t)$ minus the outgoing heat $\phi(x + \delta x, t)$, such that we obtain

$$\frac{\partial Q}{\partial t}\delta x = \phi(x, t) + f(x)\,\delta x - \phi(x + \delta x, t).$$

Dividing by δx and **taking the limit** $\delta x \to 0$ thus yields

$$\frac{\partial Q}{\partial t} = -\frac{\partial \phi}{\partial x}(x, t) + f(x).$$

Combining with **Fourier's law**

$$\phi(x, t) = -k\frac{\partial u}{\partial x}(x, t)$$

where k is the thermal conductivity, we obtain the equation

$$c\rho \, \frac{\partial u}{\partial t}(x, t) = k\frac{\partial^2 u}{\partial x^2}(x, t) + f(x) \qquad x \in (a, b),\ t > 0$$

which is the **heat equation** in one dimension, a **partial differential equation**. The heat equation describes the **evolution of the temperature in a system** (here the metal bar) and has as its unknown the temperature distribution $u(x, t)$ at any location at any point in time.

To simplify matters in this lecture **we restrict** ourselves to the case of the **stationary boundary value problems**, i.e. where all time derivatives vanish. In the case of the heat equation we would thus assume the **temperature** $u(x, t)$ to be in a state where it **no longer changes with time**, i.e. $\frac{\partial u}{\partial t} = 0$. Dropping thus the zero terms and the dependency on t we obtain the **stationary heat equation**

$$-k\,u''(x) = f(x) \qquad x \in (a,b).$$

where we used the common notation $u'' = \frac{\partial^2 u}{\partial x^2}$ for spatial derivatives.

To fully specify the problem and find a unique solution we still **need to indicate what happens at the extremities of the bar**, i.e. at $u(a)$ and at $u(b)$. Since these two points sit at the boundary of our computational domain $[a,b]$ we usually call these conditions on u **boundary conditions**.

One simple type of boundary conditions is to just set these extremal points to some values, i.e. enforce $u(a) = \gamma_a$ and $u(b) = \gamma_b$. Physically this implies a condition where the bar is in contact with heat baths of constant temperature, such that the temperature values at the boundary are known. We obtain the **heat equation using Dirichlet boundary conditions**:

$$\begin{cases} -k\,u''(x) = f(x) & x \in (a,b). \\ u(a) = \gamma_a \\ u(b) = \gamma_b \end{cases}$$

Note that other types of boundary conditions are possible, see for example chapter 10.1 of Driscoll, Brown: Fundamentals of Numerical Computation for a more general treatment.

For completeness we now provide the general definition of a boundary value problem (BVP) before specialising to the case we will focus on in this course.

Definition: Boundary value problems (general definition)

Given functions ϕ, g_a and g_b as well as a problem domain $[a,b]$ a boundary-value problem is the set of equations

$$\begin{cases} u''(x) = \phi(x, u, u'') & x \in (a,b), \\ g_a(u(a), u'(a)) = 0, \\ g_b(u(b), u'(b)) = 0 \end{cases}$$

to be solved for the unknown function $u : [a,b] \to \mathbb{R}$.

Notice that ϕ thus takes as input a point x, the *entire* function u and the *entire* function u'.

The functions g_a and g_b take two values, namely the function value and the derivative value at the boundary points a, respectively b. These functions are called **boundary conditions**.

To simplify matters for this course we will make two restrictions to this setting:

- We will only consider **linear boundary value problems**, that is problems where the the functions ϕ, g_a and g_b are linear in the arguments involving u and u'. Specifically this means that $\phi(x, u, u') = -p(x)\,u'(x) - q(x)\,u(x) + r(x)$, where p, q and r are known functions from $[a, b]$ to $\mathbb{R}$.
- We will only consider **Dirichlet boundary conditions**, that is cases where $g_a(u(a), u'(a)) = u(a) - \gamma_a$ and $g_b(u(b), u'(b)) = u(b) - \gamma_b$ where $\gamma_a, \gamma_b \in \mathbb{R}$.

Definition: Linear Dirichlet boundary value problem (our setting)

Given a domain $[a, b]$ and functions $p : [a, b] \to \mathbb{R}$, $q : [a, b] \to \mathbb{R}$, $r : [a, b] \to \mathbb{R}$ as well as boundary values $\gamma_a, \gamma_b \in \mathbb{R}$ find the function $u : [a, b] \to \mathbb{R}$ such that

$$\begin{cases} u''(x) + p(x)\,u'(x) + q(x)\,u(x) = r(x) & x \in (a, b), \\ u(a) = \gamma_a \\ u(b) = \gamma_b \end{cases} \tag{1}$$

In line with the discussion at the beginning of this notebook, we recover the stationary heat equation from equation (1) by setting $p(x) = 0$, $q(x) = 0$ and $r(x) = -f(x)/k$, resulting in the problem

$$\begin{cases} u''(x) = -\dfrac{1}{k} f(x) & x \in (a, b). \\ u(a) = \gamma_a,\ u(b) = \gamma_b \end{cases} \tag{2}$$

```
# TODO("Sketch also how the Poisson equation and the diffusion equation arise.")
```

Differentiation matrices

Having introduced the problem setting of boundary value problems (1), we will now build a numerical scheme for solving such problems in practice.

Similar to our discussion in the previous chapters on Interpolation, Numerical integration or Numerical differentiation we first **perform a discretisation**: Instead of solving the problem on all of the continous interval $[a, b]$ we discretise the interval into N subintervals. Taking N pieces of equal length $h = \frac{b-a}{N}$ leads to the nodes

$$x_i = a + i\,h, \qquad \forall i = 0, \dots, N, \qquad h = \frac{b-a}{N}. \tag{3}$$

Note, that here our node indexing starts at $\mathbf{0}$.

Since the functions p, q and r are given to us, we can easily compute discretised representations

$$p_i = p(x_i), \quad q_i = q(x_i), \quad r_i = r(x_i), \qquad \forall i = 0, \dots, N,$$

which we can denote as the vectors, for example

$$\mathbf{r} = \begin{pmatrix} r_0 \\ r_1 \\ \vdots \\ r_N \end{pmatrix} = \begin{pmatrix} r(x_0) \\ r(x_1) \\ \vdots \\ r(x_N) \end{pmatrix} \in \mathbb{R}^{n+1}.$$

In analogy, **solving a boundary value problem (1) in the discretised setting** boils down to **finding a vector $\mathbf{u} \in \mathbb{R}^{n+1}$** such that its **entries u_i are approximations to $u(x_i)$** for $i = 0, \dots, N$, i.e. the exact solution u evaluated at the nodes x_i. Before we discuss in details how to do this (see the section on the collocation method below), we first need to understand how one can express the derivative $u'(x_i)$ and $u''(x_i)$ in terms of the values $u(x_i)$.

To answer this point, let us consider a general function f for which we know the values $f(x_i)$ on the nodal points for $i = 0, \dots, N$. Our goal is to find a vector $\mathbf{g}$ such that $g_i \approx f'(x_i)$, i.e. we have a **representation of the derivative of f on the nodal points**. From chapter 09 on Numerical differentiation we already know the *forward* finite difference formula, which suggests

$$g_i = D_h^+ f(x_i) = \frac{f(x_i + h) - f(x_i)}{h} = \frac{f(x_{i+1}) - f(x_i)}{h} = \frac{f_{i+1} - f_i}{h} \qquad \text{for } i = 0, \dots, N$$

Notably we cannot apply this formula to $i = N$ because then the formula would refer to the value f_{N+1}, which we do not know. However, in this setting the *backwards* finite-difference formula applies:

$$g_N = D_h^- f(x_N) = \frac{f(x_N) - f(x_N - h)}{h} = \frac{f_N - f_{N-1}}{h}$$

Collecting the function values of f in a vector

$$\mathbf{f} = \begin{pmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_N) \end{pmatrix} \in \mathbb{R}^{N+1}$$

leads to the vector equation

$$\begin{pmatrix} f'(x_0) \\ f'(x_1) \\ \vdots \\ f'(x_N) \end{pmatrix} \approx \mathbf{g} = \widetilde{\mathbf{D}}_x \mathbf{f}, \qquad \widetilde{\mathbf{D}}_x = \frac{1}{h} \begin{pmatrix} -1 & 1 & & & \\ & -1 & 1 & & \\ & & \ddots & \ddots & \\ & & & -1 & 1 \\ & & & -1 & 1 \end{pmatrix}.$$

Notice that for the square matrix $\widetilde{\mathbf{D}}_x \in \mathbb{R}^{(N+1)\times(N+1)}$ all elements we do not show are zero.

Here, the matrix $\widetilde{\mathbf{D}}_x \in \mathbb{R}^{(N+1)\times(N+1)}$ provides a discrete approximation of the derivative operator $\frac{d}{dx}$: it maps a vector $\mathbf{f}$ (representing a discretised form of the function f) to a vector $\mathbf{g}$, which represents a discretised form of the derivative f'.

In the construction of the matrix $\widetilde{\mathbf{D}}_x$ we used only first-order finite-difference formulas. As a result $\widetilde{\mathbf{D}}_x$ only provides a first-order approximation to the derivative operator.

A more accurate construction is to employ a second-order finite difference formula wherever possible and fall back to the second-order forward / second-order backward formulas on the boundary points. This leads to

Definition: Second-order differentiation matrix for first derivative

A second-order differentiation matrix for the **first** derivative is

$$\mathbf{D}_x = \frac{1}{h} \begin{pmatrix} -\frac{3}{2} & 2 & -\frac{1}{2} & & \\ -\frac{1}{2} & 0 & \frac{1}{2} & & \\ & \ddots & \ddots & \ddots & \\ & & -\frac{1}{2} & 0 & \frac{1}{2} \\ & & \frac{1}{2} & -2 & \frac{3}{2} \end{pmatrix} \in \mathbb{R}^{(N+1)\times(N+1)}$$

where the nodes are given in equation (3).

For solving boundary value problems (1) we not only need to be able to numerically compute first derivatives, but we also need to be able to compute second derivatives as well, that is we seek a differentiation matrix $\mathbf{D}_{xx}$ such that

$$\begin{pmatrix} f''(x_0) \\ f''(x_1) \\ \vdots \\ f''(x_N) \end{pmatrix} \approx \mathbf{D}_{xx} \begin{pmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_N). \end{pmatrix}$$

Using a second-order central finite difference scheme where possible and else a second-order one-sided scheme gives:

Definition: Second-order differentiation matrix for second derivative

A second-order differentiation matrix for the **second** derivative is

$$\mathbf{D}_{xx} = \frac{1}{h^2} \begin{pmatrix} 2 & -5 & 4 & -1 & & \\ 1 & -2 & 1 & & & \\ & 1 & -2 & 1 & & \\ & & \ddots & \ddots & \ddots & \\ & & & 1 & -2 & 1 \\ & & -1 & 4 & -5 & 2 \end{pmatrix} \in \mathbb{R}^{(N+1)\times(N+1)}$$

where the nodes are given in equation (3).

An implementation returning the second-order matrices $\mathbf{D}_x$ and $\mathbf{D}_{xx}$ for approximating $\frac{d}{dx}$ and $\frac{d^2}{dx^2}$ is given below:

```
diffmats (generic function with 1 method)
```

```julia
function diffmats(N, a, b)
    # N:  Number of subintervals for discretisation, i.e. there will be N+1 nodes
    # a:  Starting point of the computational domain [a, b]
    # b:  End point of the computational domain

    h = (b-a) / N
    x = [a + i*h for i in 0:N]  # Nodes

    # Fill Dₓ with central finite difference formula (correct for most rows)
    dx_sub   = fill(-1/2 / h, N)  # Subdiagonal   of Dₓ
    dx_super = fill( 1/2 / h, N)  # Superdiagonal of Dₓ
    Dₓ = diagm(-1 => dx_sub, 1 => dx_super)

    # Adjust first and last row of Dₓ
    Dₓ[1, 1:3]         = [-3/2,  2, -1/2] / h  # Fix first row (forward  finite diff)
    Dₓ[end, end-2:end] = [ 1/2, -2,  3/2] / h  # Fix last  row (backward finite diff)

    # Fill Dₓₓ with central finite differences (correct for most rows)
    dxx_main = fill(-2 / h^2, N+1)  # Main diagonal  of Dₓₓ
    dxx_side = fill( 1 / h^2, N)    # Side diagonals of Dₓₓ
    Dₓₓ = diagm(-1 => dxx_side, 0 => dxx_main, 1 => dxx_side)

    # Adjust first and last row of Dₓₓ
    Dₓₓ[1,   1:4]       = [2, -5, 4, -1] / h^2  # First row (forward  finite diff)
    Dₓₓ[end, end-3:end] = [-1, 4, -5, 2] / h^2  # Last row  (backward finite diff)

    (; x, Dₓ, Dₓₓ, h)
end
```

Example: Using differentiation matrices

In this example we want to test the first-order and second-order differentiation matrices D_x and D_{xx}. We consider the following function

$$f_{\text{exp}} = x + e^{\sin(4x)}$$

on the interval $[-1, 1]$. The exact first and second derivatives are

$$f'_{\text{exp}}(x) = 1 + 4\cos(4x)e^{\sin(4x)}$$
$$f''_{\text{exp}}(x) = 4(4\cos^2(4x) - 4\sin(4x))e^{\sin(4x)}$$

We start by discretising on 20 equispaced nodes (i.e. $N = 19$ subintervals):

```
begin
    fexp(x)      = x + exp( sin(4x) )
    dx_fexp(x)   = 1 + 4cos(4x) * exp( sin(4x) )  # Reference derivative
    dx2_fexp(x) = 4 * ( 4cos(4x)^2 - 4sin(4x) ) * exp( sin(4x) )  # and 2nd deriv.
end;
```

```
(x = [-1.0, -0.894737, -0.789474, -0.684211, -0.578947, -0.473684, -0.368421, -0.263158, -
```

```
t, Dₓ, Dₓₓ = diffmats(19, -1, 1)
```

and we evaluate f_{exp} on the nodes:

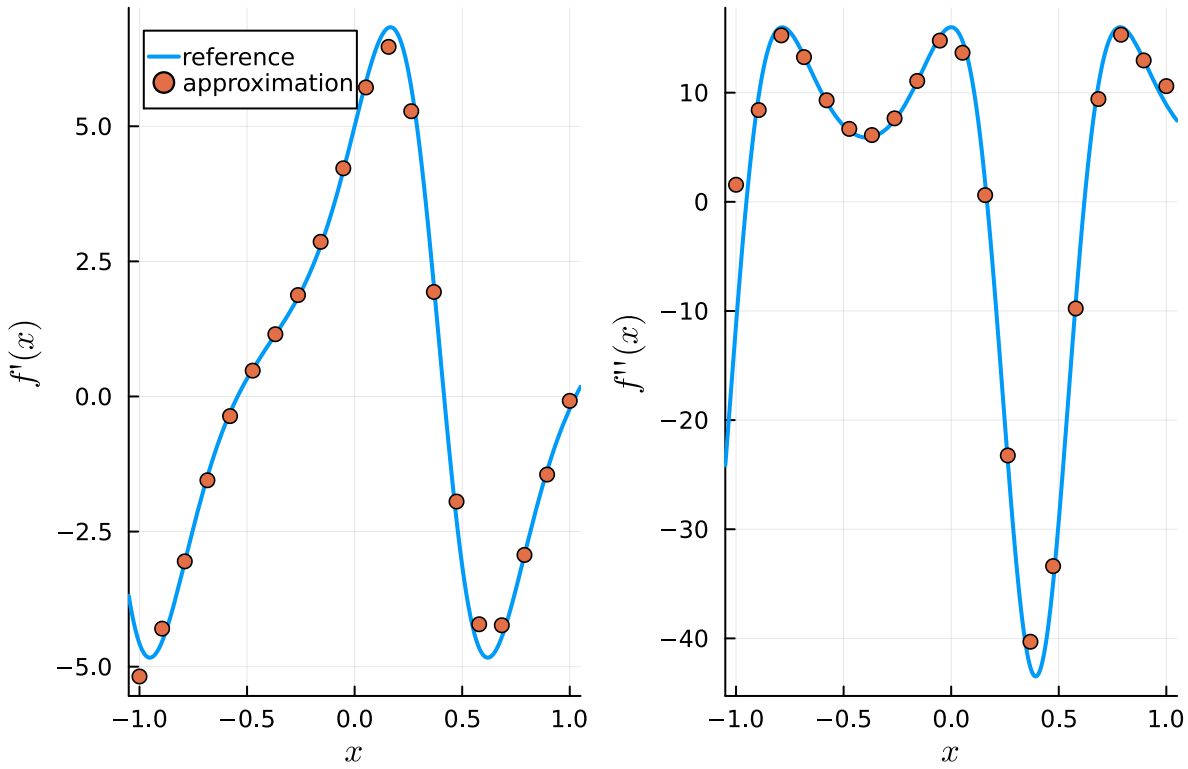
```
fexp_t =
[1.13145, 0.632629, 0.226961, -0.00971449, -0.0994863, -0.0861645, 0.00119581, 0.156326, (
```

```
fexp_t = fexp.(t)
```

Next we obtain approximate derivatives using the matrix-vector multiplication with the differentiation matrices:

```
begin
    dx_fexp_t  = Dₓ   * fexp_t
    dx2_fexp_t = Dₓₓ  * fexp_t
end;
```

Comparing against the reference values shows a rather poor accuracy:




```julia
let
    p = plot(dx_fexp, xlims=[-1.05, 1.05], xaxis=L"x", yaxis=L"f'(x)", label="reference", lw=2, legend=:topleft)
    p = scatter!(p, t, dx_fexp_t, label="approximation")

    q = plot(dx2_fexp, xlims=[-1.05, 1.05], xaxis=L"x", yaxis=L"f''(x)", label="reference", lw=2, legend=false)
    q = scatter!(q, t, dx2_fexp_t, label="approximation")

    plot(p, q)
end
```

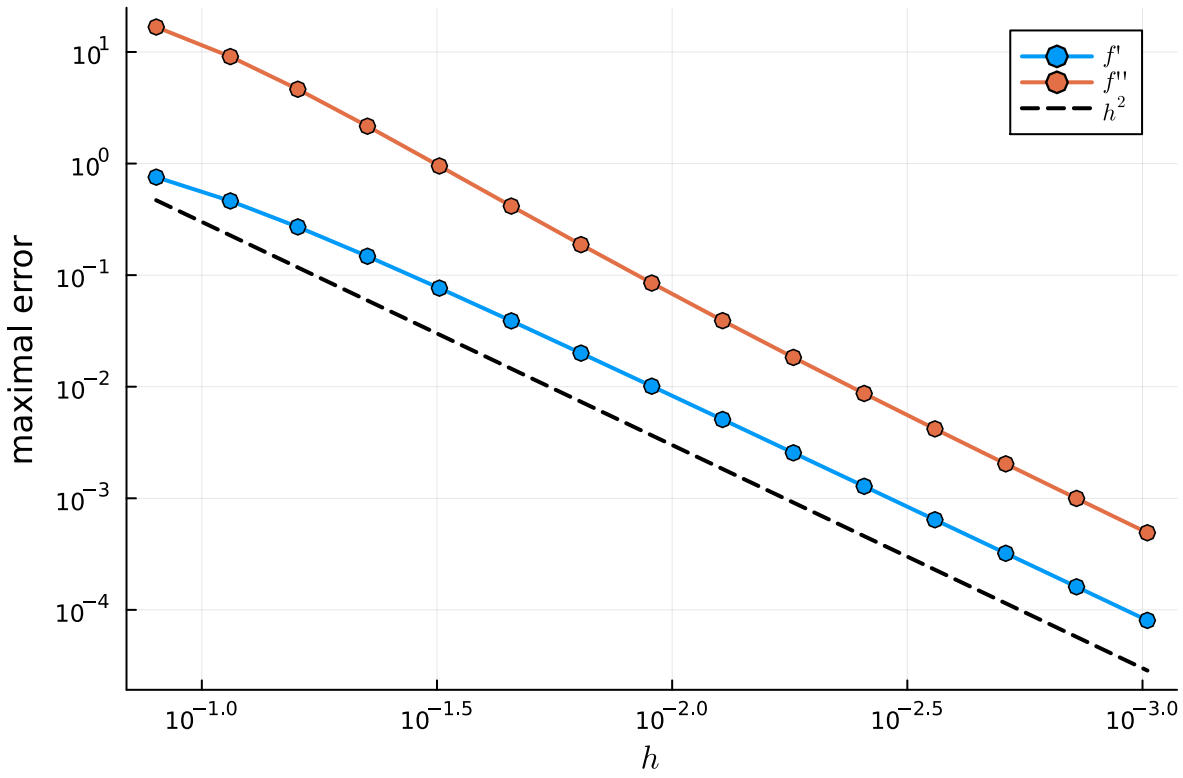
Since we took a second-order finite difference formula, we would expect a second-order convergence. Plotting the maximal error of the approximated derivative values `dx_fexp_t` respectively `dx2_fexp_t`, that is

```julia
maximum(abs.(dx_fexp.(t)  - dx_fexp_t))
```

and

```julia
maximum(abs.(dx2_fexp.(t) - dx2_fexp_t))
```

against the subinterval size h on a log-log plot confirms this:




```julia
let
    Ns = round.(Int, 2.0 .^ (4:0.5:11))
    hs = zeros(length(Ns))
    error_df  = zeros(length(Ns))
    error_df2 = zeros(length(Ns))
    for k in 1:length(Ns)
        t, Dₓ, Dₓₓ, h = diffmats(Ns[k], -1, 1)
        fexp_t = fexp.(t)
        error_df[k]  = maximum(abs.(dx_fexp.(t)  - Dₓ  * fexp_t))
        error_df2[k] = maximum(abs.(dx2_fexp.(t) - Dₓₓ * fexp_t))
        hs[k] = h
    end

    p = plot(; xaxis=:log, xlabel=L"h", yaxis=:log, ylabel="maximal error", xflip=true)
    plot!(p, hs, error_df,  mark=:o, lw=2, label=L"f'")
    plot!(p, hs, error_df2, mark=:o, lw=2, label=L"f''")

    # Guide line
    plot!(p, hs, 30 * hs .^ 2, lw=2, ls=:dash, c=:black, label=L"h^2")

    xticks!(p, 10 .^ (-1:-0.5:-3))
    yticks!(p, 10 .^ (1:-1.0:-4))

    p
end
```

Collocation method

We will now construct a numerical method based on the finite differences differentiation matrices for solving the linear boundary value problem

$$\begin{cases} u''(x) + p(x)\,u'(x) + q(x)\,u(x) = r(x) & x \in (a,b), \\ u(a) = \gamma_a \\ u(b) = \gamma_b \end{cases}$$

with equispaced nodes

$$x_i = a + i\,h, \qquad \forall i = 0, \ldots, N, \qquad h = \frac{b-a}{N}.$$

Rather than solving for the function $u : [a,b] \to \mathbb{R}$ we will instead seek the vector $\tilde{\mathbf{u}} \in \mathbb{R}^{n+1}$ of its approximate function values at the nodes x_i:

$$\tilde{\mathbf{u}} = \begin{pmatrix} \tilde{u}_0 \\ \tilde{u}_1 \\ \vdots \\ \tilde{u}_N \end{pmatrix} \approx \begin{pmatrix} u(x_0) \\ u(x_1) \\ \vdots \\ u(x_N) \end{pmatrix}.$$

The key idea of **collocation** is to use the BVP equation (1) evaluated *at the same nodal points* x_i to impose additional constraints on the solution vector. Concretely we use the differentiation matrices to approximate derivatives of the solution, i.e.

$$\begin{pmatrix} u'(x_0) \\ u'(x_1) \\ \vdots \\ u'(x_N) \end{pmatrix} \approx \tilde{\mathbf{u}}' := \mathbf{D}_x \tilde{\mathbf{u}} \quad \text{and} \quad \begin{pmatrix} u''(x_0) \\ u''(x_1) \\ \vdots \\ u''(x_N) \end{pmatrix} \approx \tilde{\mathbf{u}}'' := \mathbf{D}_{xx} \tilde{\mathbf{u}} \tag{4}$$

With these definitions the discrete form of the first line of equation (1) therefore becomes

$$\tilde{\mathbf{u}}'' + \mathbf{P}\tilde{\mathbf{u}}' + \mathbf{Q}\tilde{\mathbf{u}} = \mathbf{r}$$

where

$$\mathbf{P} = \begin{pmatrix} p(x_0) & & \\ & \ddots & \\ & & p(x_N) \end{pmatrix}, \quad \mathbf{Q} = \begin{pmatrix} q(x_0) & & \\ & \ddots & \\ & & q(x_N) \end{pmatrix}, \quad \mathbf{r} = \begin{pmatrix} r(x_0) \\ \vdots \\ r(x_N) \end{pmatrix}.$$

Using equation (4) we can rewrite this as

$$\mathbf{L}\tilde{\mathbf{u}} = \mathbf{r} \quad \text{where} \quad \mathbf{L} = \mathbf{D}_{xx} + \mathbf{P}\mathbf{D}_x + \mathbf{Q} \tag{5}$$

where we note $\mathbf{L} \in \mathbb{R}^{(N+1)\times(N+1)}$, i.e. this is a linear system with $N+1$ equations and $N+1$ unknowns. Notice further that since $\mathbf{D}_{xx}$, $\mathbf{D}_x$, $\mathbf{P}$, $\mathbf{Q}$ are all sparse, banded matrices also $\mathbf{L}$ is sparse and banded.

The final step is to notice that the boundary conditions (last two lines of equation (1)) take the form of two additional conditions

$$u_0 = \gamma_a, \qquad u_N = \gamma_b,$$

which are so far not yet taken into account in equation (5). The easiest way to achieve this is to replace the first and last equation in the linear system of equation (5) exactly by these two equations given by the boundary conditions. To make this clear, let us first write equation (5) more explicitly as

$$\begin{pmatrix} L_{00} & L_{01} & \cdots & L_{0N} \\ L_{10} & L_{11} & \cdots & L_{1N} \\ \vdots & \vdots & \ddots & \vdots \\ L_{N-1,0} & L_{N-1,1} & \cdots & L_{N-1,N} \\ L_{N0} & L_{N1} & \cdots & L_{NN} \end{pmatrix} \begin{pmatrix} \tilde{u}_0 \\ \tilde{u}_1 \\ \vdots \\ \tilde{u}_{N-1} \\ \tilde{u}_N \end{pmatrix} = \begin{pmatrix} r_0 \\ r_1 \\ \vdots \\ r_{N-1} \\ r_N \end{pmatrix}.$$

Now we replace the first equation by $u_0 = \gamma_a$ and the last equation by $u_N = \gamma_b$, this yields

$$\begin{pmatrix} 1 & 0 & \cdots & 0 \\ L_{10} & L_{11} & \cdots & L_{1N} \\ \vdots & \vdots & \ddots & \vdots \\ L_{N-1,0} & L_{N-1,1} & \cdots & L_{N-1,N} \\ 0 & 0 & \cdots & 1 \end{pmatrix} \begin{pmatrix} \tilde{u}_0 \\ \tilde{u}_1 \\ \vdots \\ \tilde{u}_{N-1} \\ \tilde{u}_N \end{pmatrix} = \begin{pmatrix} \gamma_a \\ r_1 \\ \vdots \\ r_{N-1} \\ \gamma_b \end{pmatrix}.$$

Finally we make the notation more compact by introducing the $(N-1) \times (N+1)$ matrix

$$\mathbf{E} = \left(\begin{array}{c|ccc|c} 0 & 1 & & & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & & & 1 & 0 \end{array}\right)$$

which is constructed such that the product $\mathbf{EL}$ of this matrix with an $(N+1) \times (N+1)$ matrix $\mathbf{L}$ deletes the first and last rows of $\mathbf{L}$. Similarly $\mathbf{Er}$ deletes the first and last rows of a matrix $\mathbf{r}$. Let us demonstrate this on a 4×4 example:

```
E_2by4 = 2×4 Matrix{Int64}:
          0  1  0  0
          0  0  1  0
```

```
E_2by4 = [0 1 0 0;
          0 0 1 0]
```

```
M = 4×4 Matrix{Int64}:
      0   1   2   3
     10  11  12  13
     20  21  22  23
     30  31  32  33
```

```
M = [00 01 02 03;
     10 11 12 13;
     20 21 22 23;
     30 31 32 33]
```

```
2×4 Matrix{Int64}:
 10  11  12  13
 20  21  22  23
```

```
E_2by4 * M
```

Using $\mathbf{E}$ we can write the linear system more compactly as

$$\underbrace{\left(\begin{array}{ccccc} 1 & 0 & \cdots & 0 & 0 \\ \hline & & \mathbf{EL} & & \\ \hline 0 & 0 & \cdots & 0 & 1 \end{array}\right)}_{=\mathbf{A}} \tilde{\mathbf{u}} = \underbrace{\left(\begin{array}{c} \gamma_a \\ \hline \mathbf{Er} \\ \hline \gamma_b \end{array}\right)}_{=\mathbf{b}}. \tag{6}$$

This is an $(N+1) \times (N+1)$ linear system $\mathbf{Au} = \mathbf{b}$ with unknown $\mathbf{u}$ and — as we will observe in more detail below — a sparse, banded matrix $\mathbf{A}$. Problem (6) can therefore be **efficiently solved** using direct methods based on (sparse) LU factorisation (chapter 5) or an iterative approaches (chapter 6).

Example: Setting up a collocation linear system

As an example we discretise the boundary value problem

$$u'' = u - xu', \quad u(0) = -2, \quad u(2) = 3.$$

- **Step 1: Selection of nodes:** We choose an equispaced grid with $N = 4$ subintervals. As a result we have the following discretisation nodes

$$h = \frac{2-0}{4} = \frac{1}{2} \quad \Rightarrow \quad x_0 = 0, x_1 = \frac{1}{2}, x_2 = 1, x_3 = \frac{3}{2}, x_4 = 2$$

- **Step 2: Setup of operator L:** Bringing the differential equation into the form of equation (1), i.e. $u'' + xu' - u = 0$ we notice $p(x) = x$, $q(x) = -1$ and $r(x) = 0$. Therefore

$$\mathbf{P} = \begin{pmatrix} 0 & & & & \\ & 1/2 & & & \\ & & 1 & & \\ & & & 3/2 & \\ & & & & 2 \end{pmatrix}, \quad \mathbf{Q} = -\begin{pmatrix} 1 & & & & \\ & 1 & & & \\ & & 1 & & \\ & & & 1 & \\ & & & & 1 \end{pmatrix}, \quad \mathbf{r} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Using the second-order differentiation matrices $\mathbf{D}_x$ and $\mathbf{D}_{xx}$ we therefore obtain:

$$\begin{aligned} \mathbf{L} &= \mathbf{D}_{xx} + \mathbf{P}\mathbf{D}_x + \mathbf{Q} \\ &= \frac{1}{(1/2)^2} \begin{pmatrix} 2 & -5 & 4 & -1 & \\ 1 & -2 & 1 & & \\ & 1 & -2 & 1 & \\ & & 1 & -2 & 1 \\ & -1 & 4 & -5 & 2 \end{pmatrix} + \mathbf{P}\frac{1}{1/2} \begin{pmatrix} -\frac{3}{2} & 2 & -\frac{1}{2} & & \\ -\frac{1}{2} & 0 & \frac{1}{2} & & \\ & -\frac{1}{2} & 0 & \frac{1}{2} & \\ & & -\frac{1}{2} & 0 & \frac{1}{2} \\ & & \frac{1}{2} & -2 & \frac{3}{2} \end{pmatrix} + \mathbf{Q} \\ &= \begin{pmatrix} 7 & -20 & 16 & -4 & 0 \\ 7/2 & -9 & 9/2 & 0 & 0 \\ 0 & 3 & -9 & 5 & 0 \\ 0 & 0 & 5/2 & -9 & 11/2 \\ 0 & -4 & 18 & -28 & 13 \end{pmatrix} \end{aligned}$$

- **Step 3: Application of boundary conditions:** To apply the boundary conditions we change the first and last lines of the discretised differential equation $\mathbf{Lu} = \mathbf{r}$ arriving at the linear system $\mathbf{Au} = \mathbf{b}$ with

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 7/2 & -9 & 9/2 & 0 & 0 \\ 0 & 3 & -9 & 5 & 0 \\ 0 & 0 & 5/2 & -9 & 11/2 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \qquad \mathbf{b} = \begin{pmatrix} -2 \\ 0 \\ 0 \\ 0 \\ 3 \end{pmatrix}$$

- **Step 4: Solution of linear system (6):** The collocated system $\mathbf{Au} = \mathbf{b}$ we can now solve using LU factorisation:

```
[-2.0, -0.218687, 1.11818, 2.14394, 3.0]
```

```
let
    A = [1     0  0     0  0;
         7/2  -9  9/2   0  0;
         0     3  -9    5  0;
         0     0  5/2  -9  11/2;
         0     0  0     0    1]
    b = [-2;
          0;
          0;
          0;
          3]
    A \ b
end
```

Finally we present a generic implementation of the collocation method for solving linear boundary value problems based on the `diffmats` function and LU factorisation to solve the arising linear system:

```
fd_bvp (generic function with 1 method)
```

```julia
function fd_bvp(N, a, b, p, q, r, γa, γb)
    # Solves the differential equation
    #   u''(x) + p(x) u'(x) + q(x) u(x) = r(x)
    # in the domain [a, b] with Dirichlet boundary conditions u(a) = γa, u(b) = γb
    #
    # N:  Number of subintervals for discretisation, i.e. there will be N+1 nodes
    # a:  Starting point of the computational domain [a, b]
    # b:  End point of the computational domain
    # p, q, r:  Julia functions to define the unknowns in the equation above
    # γa, γb:   Dirichlet boundary values

    (; x, Dₓ, Dₓₓ, h) = diffmats(N, a, b)

    # Assemble discretised differential equation operator L
    P = Diagonal(p.(x))
    Q = Diagonal(q.(x))
    L = Dₓₓ + P * Dₓ + Q

    # Apply bounary conditions
    A = zeros(N+1, N+1)
    A[2:end-1, :] = L[2:end-1, :]  # Set, but exclude first and last row of L
    A[1, 1] = A[end, end] = 1      # Set upper left and lower-right corners

    b = zeros(N+1)
    b[2:end-1] = r.(x[2:end-1])  # Set, but exclude first and last entry
    b[1]   = γa  # Set first entry
    b[end] = γb  # Set last entry

    # Solve the linear system
    u = A \ b

    (; x, u, h, A, b, Dₓ, Dₓₓ)
end
```

To test this implementation we consider the following heat equation

$$\begin{cases} -u''(x) = \sin(x) & x \in (0, 2\pi) \\ u(0) = 1, \quad u(2\pi) = 2, \end{cases}$$

which is to be solved in the interval $[0, 2\pi]$.

Rewriting this equation as $u''(x) = -\sin(x)$ and comparing with the interface of the function `fd_bvp` we notice the following parameters:

```julia
begin
    p(x) = 0.0
    q(x) = 0.0
    r(x) = -sin(x)

    a = 0.0  # Interval begin
    b = 2π   # Interval end

    γa = 1   # Left  boundary value
    γb = 2   # Right boundary value
end;
```

In this case the exact solution can be found analytically as

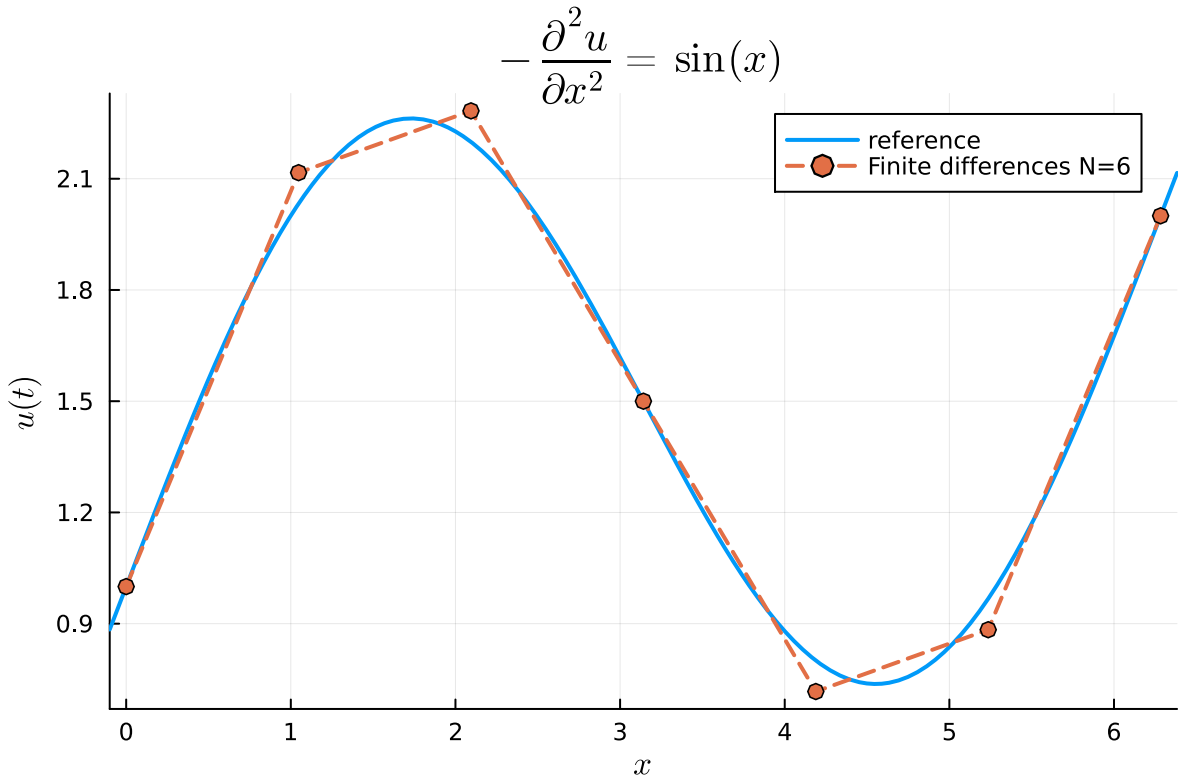
```
u_exact (generic function with 1 method)
```

```julia
u_exact(x) = sin(x) + x/2π + 1
```

We solve this using finite differences

```julia
res_fd = fd_bvp(N, a, b, p, q, r, γa, γb);
```

and plot the result:




```
let
    plot(u_exact; xlims=(a-0.1, b+0.1), legend=:topright,
         label="reference", lw=2, xlabel=L"x", ylabel=L"u(t)",
         title=L"-\frac{\partial^2 u}{\partial x^2} = \sin(x)")

    plot!(res_fd.x, res_fd.u; label="Finite differences N=$N", mark=:o, lw=2,
    ls=:dash)
end
```

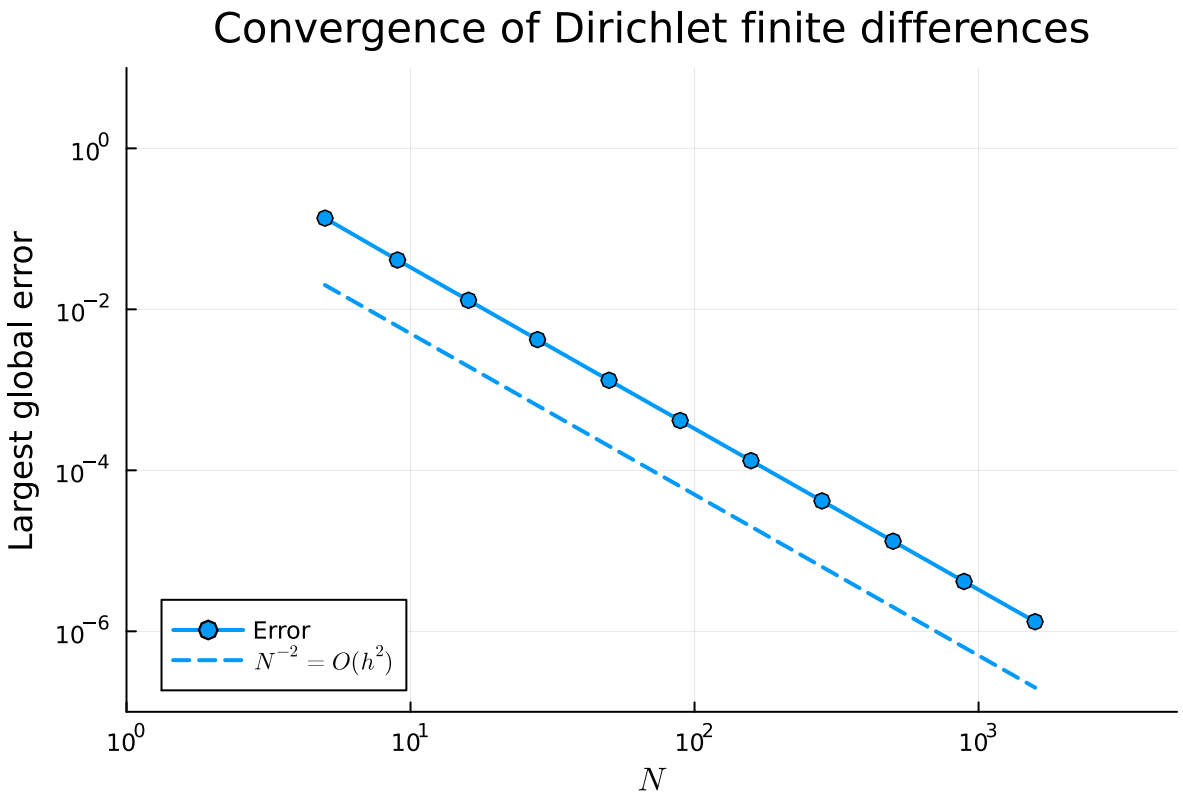
N = 6

h = 1.05

We observe that as the number of points `N` is increased, the solution visually becomes more and more accurate. The output of our numerical procedure are the computed points $u_0, u_1, \ldots u_N$. These points approximate the true function values $u(x_0), u(x_1), \ldots, u(x_N)$ of u evaluated at the nodal points. A measure for the accuracy of our obtained solution is thus the maximal deviation any of these computed points has from from the true $u(x_i)$. This measure is called the **global error** defined by

$$|e_i| = |u(x_i) - u_i| \qquad \text{for } i = 1, \ldots, N.$$

Since we used a quadratic difference formulas for building the differentiation matrices $\boldsymbol{D}_{\mathrm{x}}$ and $\boldsymbol{D}_{\mathrm{xx}}$ we obtain **quadratic convergence** in this global error with increasing $\boldsymbol{N}$:




```julia
let
    Ns = [ round(Int, 5 * 10^k) for k in 0:0.25:2.5 ]
    errors = Float64[]
    for N in Ns
        res_fd = fd_bvp(N, a, b, p, q, r, ya, yb)
        x = res_fd.x
        u = res_fd.u
        error = [ u_exact(x[j]) - u[j] for j in 1:length(x)]
        push!(errors, maximum(error))
    end

    p = plot(; title="Convergence of Dirichlet finite differences",
               xaxis=:log, yaxis=:log,
               xlabel=L"N", ylabel="Largest global error",
               legend=:bottomleft,
               xlims=(1, 5e3), ylims=(1e-7, 10))
    plot!(p, Ns, errors; lw=2, mark=:o, c=1, label="Error")
    plot!(p, Ns, 0.02(first(Ns) ./ Ns).^2, ls=:dash, lw=2, label=L"N^{-2} = O(h^2)", c=1)

    xticks!(p, 10.0 .^ (0:1:3))
    yticks!(p, 10.0 .^ (0:-2:-6))
    p
end
```

In line with this discussion we define convergence for numerical schemes for boundary value problems as follows:

Definition: Convergence order for boundary value problems

A numerical scheme approximating a boundary value problem (9) **converges with order p** if there exists a constant $\alpha > 0$ such that

$$\max_{j=0,\ldots,N+1} |u(x_j) - u_j| \leq \alpha\, h^p$$

provided that the solution u is sufficiently regular.

Since we used a quadratic difference formulas for building the differentiation matrices D_x and D_{xx} we obtain indeed quadratic convergence $p = 2$, as is formally summarised in the following theorem.

Theorem 1: Global error of finite differences

Under the assumption that the exact solution u of the Dirichlet boundary value problem (1) is **four times differentiable**, the finite-difference scheme of equations (5) and (6) involving the **second-order differentiation matrices $\mathbf{D}_x$** and $\mathbf{D}_{xx}$ converges as

$$\max_{j=1,\ldots,N} |u(x_j) - u_j| \leq \alpha\, \|u''''\|_\infty\, h^2$$

where $\alpha > 0$ is a constant that independent of u and h. It thus achieves **quadratic convergence**.

The proof of this result goes beyond the scope of this course and can for example be found in chapter 7.4 of *Introduction to Numerical Analysis* by Stoer and Bulirsch. The key observation to take a way from this developments is:

Observation: Convergence order of BVP scheme and FD formula

Provided our solution function u can be infinitely often differentiated continously, than we can **obtain p-th order convergence** (as $h \to 0$ or $N \to \infty$) of the solution vector $\tilde{\mathbf{u}}$ obtained in (6) provided that we consistently use **p-th order finite-difference formulas** to build *both* the differentiation matrices D_x and D_{xx} in (5).

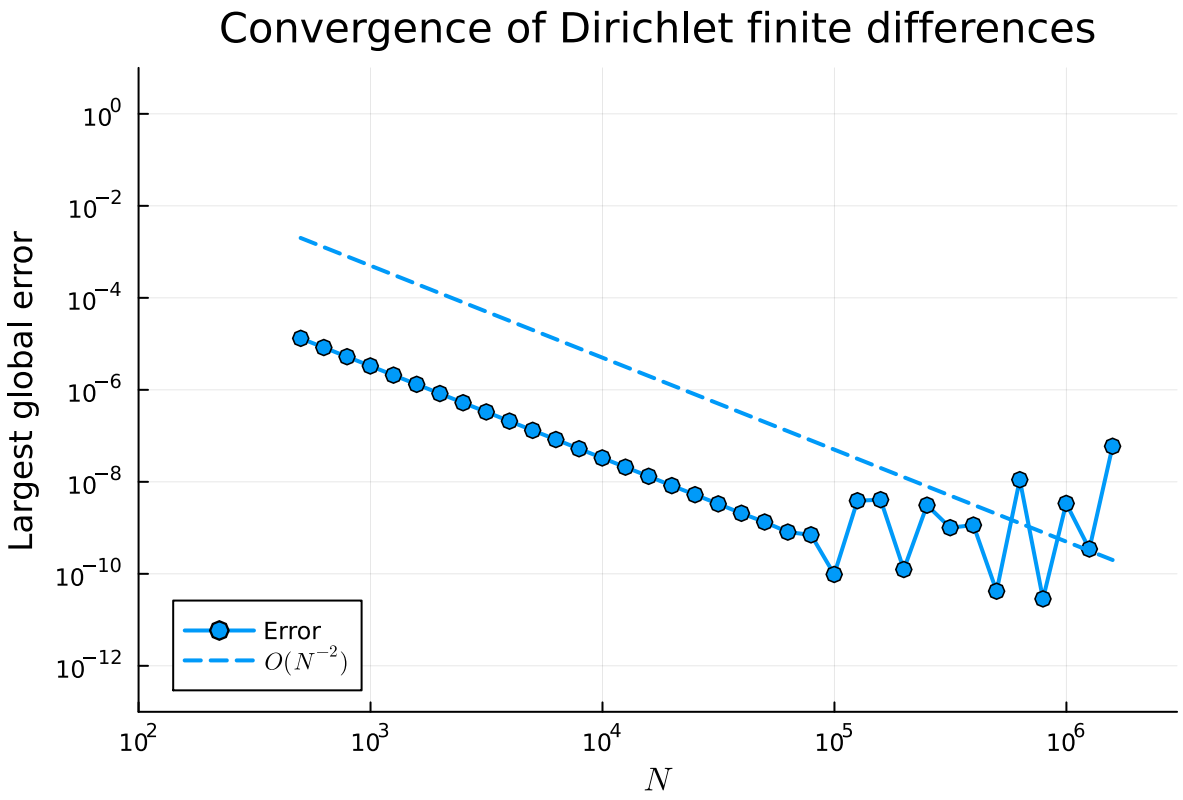
Numerical stability

We want to push the convergence plot from above a little further. For this we employ an implementation of `fd_bvp` that directly sets up the matrix `A` as a sparse matrix. Unfold the cell below to see the implementation.

```
fd_bvp_fast (generic function with 1 method)
```

```
7×7 Tridiagonal{Float64, Vector{Float64}}:
 1.0        0.0         ⋅          ⋅          ⋅          ⋅         ⋅
 0.911891  -1.82378    0.911891    ⋅          ⋅          ⋅         ⋅
  ⋅         0.911891  -1.82378    0.911891    ⋅          ⋅         ⋅
  ⋅          ⋅         0.911891  -1.82378    0.911891    ⋅         ⋅
  ⋅          ⋅          ⋅         0.911891  -1.82378    0.911891   ⋅
  ⋅          ⋅          ⋅          ⋅         0.911891  -1.82378   0.911891
  ⋅          ⋅          ⋅          ⋅          ⋅         0.0       1.0
```

```
fd_bvp_fast(N, a, b, p, q, r, γa, γb).A
```



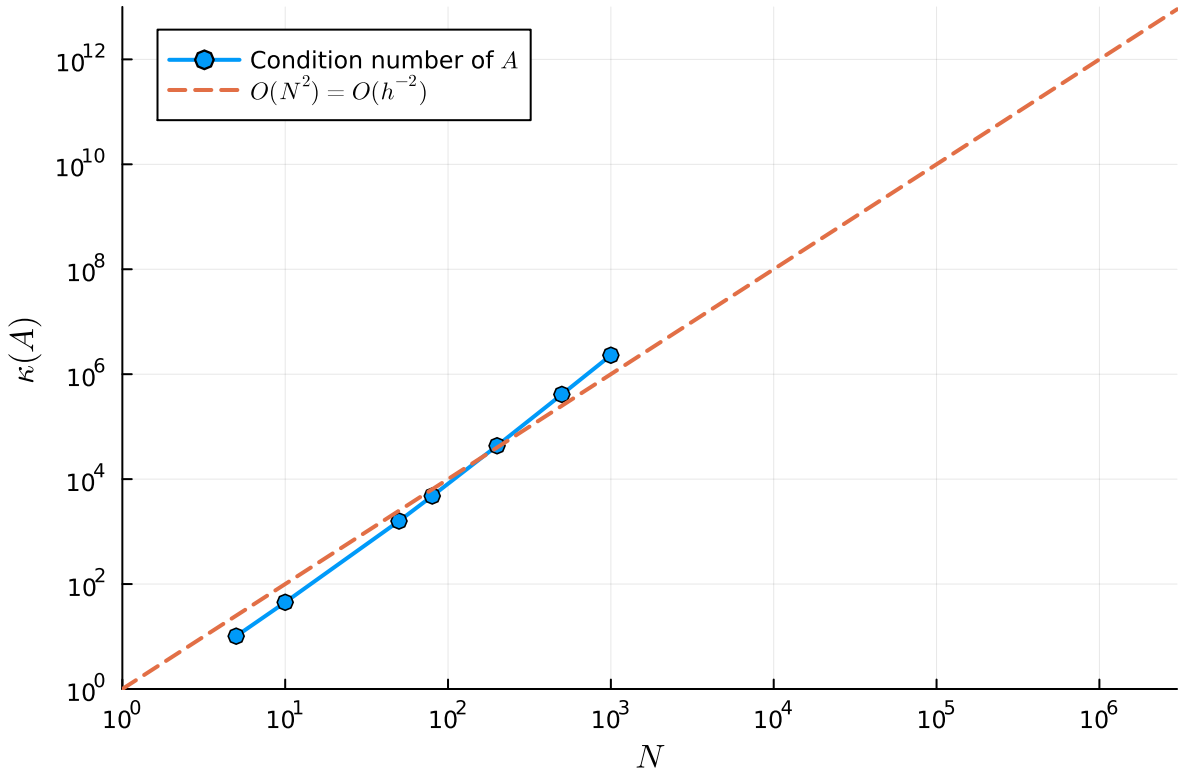

```julia
let
    Ns = [ round(Int, 5 * 10^k) for k in 2:0.1:5.5 ]
    errors = Float64[]
    for N in Ns
        res_fd = fd_bvp_fast(N, a, b, p, q, r, ya, yb)
        x = res_fd.x
        u = res_fd.u
        error = [ u_exact(x[j]) - u[j] for j in 1:length(x)]
        push!(errors, maximum(error))
    end

    p = plot(; title="Convergence of Dirichlet finite differences",
               xaxis=:log, yaxis=:log,
               xlabel=L"N", ylabel="Largest global error",
               legend=:bottomleft,
               xlims=(100, 3e6), ylims=(1e-13, 10))
    plot!(p, Ns, errors; lw=2, mark=:o, c=1, label="Error")
    plot!(Ns, 0.002(first(Ns) ./ Ns).^2, ls=:dash, lw=2, label=L"O(N^{-2})", c=1)

    xticks!(p, 10.0 .^ (2:1:6))
    yticks!(p, 10.0 .^ (0:-2:-12))
    p
end
```

While initially the convergence thus nicely follows the expected convergence curve, **for larger N the convergence degrades and the error starts increasing again**.

Similar to our discussion on numerical stability in the chapter on numerical differentiation this error plot is the result of a balance between two error contributions:

- The **discretisation error** due to the choice of N, where as N gets larger this error **decreases** as $O(N^{-2})$.
- The **error due to finite floating-point precision**, which turns out to increase as N increases.

With respect to the second error contribution the underlying problem is related to solving the linear system $\mathbf{Au} = \mathbf{b}$ in Algorithm 1. As it turns out the **condition number of the system matrix $\mathbf{A}$ grows** as $O(N^2) = O(h^{-2})$:



As a result **the finer we make the discretisation**, i.e. the **larger N**, the harder it becomes to solve the system $\mathbf{Au} = \mathbf{b}$, such that the **floating-point error in computing the solution $\mathbf{u}$ itself** increases quadratically.

To make this clear let's consider $N = 10^5$. At this stage the condition number is about 10^{10}, such that the typical 10^{-16} error we make when representing any floating-point operation (such as the computation of $\mathbf{b}$) gets amplified to a relative error in the solution $\mathbf{u}$ of around 10^{-6}: at most around 6 digits of the solution can be known exactly.

As a result around $N = 10^5$ the floating-point error becomes the leading error contribution, such that increasing N beyond 10^5 thus causes the total observed error of the numerical procedure to increase again.

⚠ TODO ⚠

Needs an adaptive pass from here.

```
TODO("Needs an adaptive pass from here.")
```

Galerkin methods

Let us return to the general heat equation using Dirichlet boundary conditions:

$$\begin{cases} -k\,\dfrac{\partial^2 u}{\partial x^2}(x) = f(x) \qquad x \in (0, L). \\ \qquad u(0) = b_0, \quad u(L) = b_L \end{cases} \tag{17}$$

We saw that **with finite differences** the increasing condition number of the system matrix $\mathbf{A}$ as we increase N makes it **impossible to obtain the solution to arbitrary accuracy**. In the above example we were unable to obtain a solution to higher accuracy than 10^{-10}, irrespective of what value for N we chose. In this section we will develop an alternative approach to solve boundary value problems, which is more general.

Let us assume we are given some smooth function $\psi(x)$, the so-called **test function**, for which we will additionally assume that it **vanishes at the boundary**, i.e. $\psi(0) = \psi(L) = 0$. If we multiply the first line of (17) by this function and integrate over the full computational domain $[0, L]$, this yields:

$$\begin{aligned} \int_0^L f(x)\psi(x)\,dx &= \int_0^L -k\,u''(x)\psi(x)\,dx \\ &\overset{(*)}{=} [-k\,u'(x)\psi(x)]_0^L + \int_0^L ku'(x)\psi'(x)\,dx \\ &= \int_0^L ku'(x)\psi'(x)\,dx \end{aligned} \tag{18}$$

where we used partial integration in step $(*)$ and the property $\psi(0) = \psi(L) = 0$ in the final step.

Let us define the **set of all test functions** as

$$V_0 = \left\{ \psi : [0, L] \to \mathbb{R} \;\middle|\; \psi \text{ smooth and } \psi(0) = \psi(L) = 0 \right\}.$$

If $u : [0, L] \to \mathbb{R}$ is a solution to (17) than our discussion implies that (18) is satisfied for all functions from V_0:

$$\begin{cases} \displaystyle\int_0^L k\, u'(x)\, \psi'(x)\, dx = \int_0^L f(x)\, \psi(x)\, dx \qquad \forall\, \psi \in V_0 \\ u(0) = b_0, \quad u(L) = b_L \end{cases} \tag{19}$$

Equation (19) is known as the **weak form** of the 1D heat equation equation (17) and solving it is interesting in its own right:

Definition: Weak form and weak solution

If a function $u : [0, L] \to \mathbb{R}$ satisfies

$$\begin{cases} \displaystyle\int_0^L k\, u'(x)\, \psi'(x)\, dx = \int_0^L f(x)\, \psi(x)\, dx \qquad \forall\, \psi \in V_0 \\ u(0) = b_0, \quad u(L) = b_L, \end{cases}$$

for all choices of the test function ψ we call u a **weak solution** of the boundary value problem (17).

The original problem (17) is additionally called the **strong form** of the heat equation BVP and its solution u the **strong solution**.

Observation: Strong and weak solutions

If u is a solution to the strong form (17) of a BVP, than it is also a solution to the weak form (19). However, **not every weak solution is also a strong solution**.

The weak form (19) looks unusual at first sight and additionally one might wonder why it is useful, since it can produce solutions, which do not satisfy the original strong problem (17).

But surprisingly for many problems from physics or engineering the situation turns out to be the reverse: it turns out that the somewhat "looser" form provided by **the weak formulation is actually more physical**. In some cases — such as the atomistic modelling of materials — using the strong form can lead to unphysical artifacts, which are in fact avoided when using the weak form.

While we do not have time to discuss this further in this course, the interested reader is referred to the master course MATH-500: Error control in scientific modelling where some of this is discussed.

Discretisation of the weak form

In order to solve (19) our goal is to rewrite the problem in the form of linear algebra — vectors and matrices. For simplicity we will only consider the case $b_0 = b_L = 0$ in this section, i.e. we restrict ourselves to the **heat equation with a zero Dirichlet boundary**

$$\begin{cases} -k\,\dfrac{\partial^2 u}{\partial x^2}(x) = f(x) \qquad x \in (0, L). \\ \qquad u(0) = u(L) = 0 \end{cases} \tag{20}$$

and **corresponding weak form**

$$\begin{cases} \displaystyle\int_0^L k\,u'(x)\,\psi'(x)\,dx = \int_0^L f(x)\,\psi(x)\,dx \qquad \forall\,\psi \in V_0 \\ \qquad u(0) = u(L) = 0. \end{cases} \tag{21}$$

One difficulty in this equation is that the condition $\forall \psi \in V_0$ corresponds to an infinite number of constraints to satisfy (as the set V_0 has infinitely many members). One idea is to approximate this condition by satisfying only finitely many constraints.

To do so we assume that ψ **can be written as a linear combination**

$$\psi(x) = \sum_{i=1}^{m} \xi_i\,\varphi_i(x), \tag{22}$$

where $\varphi_1, \varphi_2, \ldots, \varphi_m$ are a selection of m linearly independent functions from V_0, i.e. smooth functions which each satisfy $\varphi_i(0) = \varphi_i(L) = 0$. Inserting (22) into the first line of (21) leads to

$$\begin{aligned} 0 &= \int_0^L k\,u'(x)\,\psi'(x) - f(x)\psi(x)\,dx \\ &= \sum_{i=1}^{m} \xi_i \left[\int_0^L k\,u'(x)\,\varphi_i'(x) - f(x)\,\varphi_i(x)\,dx\right] \end{aligned}$$

which should be true **independent of the values of** ξ_i. As a result the above condition can be achieved if and only if the term in the **square bracket is zero all our functions** φ_i, that is if

$$\forall i = 1, \ldots, m: \qquad \int_0^L k\,u'(x)\,\varphi_i'(x)\,dx = \int_0^L f(x)\,\varphi_i(x)\,dx. \tag{23}$$

Notice, that the condition (23) is independent of the actual values taken by the coefficients $\{\xi_i\}_{i=1}^m$. Provided that (23) holds we thus do not need to worry about determining the values of $\{\xi_i\}_{i=1}^m$. In fact in all following development these coefficients will play no further role.

With this development we can replace the infinite number of constraints encoded by the $\forall \psi \in V_0$ in equation (21) by the approximate **version (23)**, which **only involves $m < \infty$ conditions** to satisfy.

Considering $u(x)$ we make an additional approximation, namely that — similar to ψ — this function can **also be approximated by a linear combination of finitely many functions**. Here, for simplicity we employ the same selection of functions, that we used for ψ leading to an ansatz

$$u(x) = \sum_{j=1}^{m} c_j \varphi_j(x). \tag{24}$$

Inserting (24) into (23) we obtain

$$\forall i = 1, \ldots, m: \qquad \int_0^L k \sum_{j=1}^{m} c_j \varphi_j'(x)\, \varphi_i'(x)\, dx = \int_0^L f(x)\, \varphi_i(x)\, dx$$

or arranged differently

$$\forall i = 1, \ldots, m: \qquad \sum_{j=1}^{m} c_j \int_0^L k\, \varphi_j'(x)\, \varphi_i'(x)\, dx = \int_0^L f(x)\, \varphi_i(x)\, dx \tag{25}$$

Since $\varphi_j(0) = \varphi_j(L) = 0$ an immediate consequence is that such a u satisfies the boundary condition $u(0) = u(L) = 0$ independent of the choice of the coefficients c_j. Solving equation (25) therefore automatically ensures that the second line of the weak problem (21) is satisfied. Additionally this equation satisfies (23) — our approximation to the first line of (21).

In summary a **solution to equation (25)** thus **satisfies** (our approximation to) **both conditions required to be a solution to the weak problem** (21). Equation (25) is thus given a special name:

Definition: Galerkin conditions

Given a basis $\varphi_1, \varphi_2, \ldots, \varphi_m$ of m linearly independent functions from the test function set V_0, the linear combination $u = \sum_{j=1}^m c_j \varphi_j$ is an approximate solution to the weak problem (21) if the coefficients $c_1, c_2, \ldots, c_m$ satisfy the **Galerkin conditions**

$$\forall i = 1, \dots, m: \qquad \sum_{j=1}^{m} c_j \int_0^L k\, \varphi_j'(x)\, \varphi_i'(x)\, dx = \int_0^L f(x)\, \varphi_i(x)\, dx \tag{25}$$

Collecting the unknown coefficients into a vector $\mathbf{c} = (c_1, c_2, \dots, c_m)^T \in \mathbb{R}^m$ and introducing the matrix $\mathbf{A} \in \mathbb{R}^{m \times m}$ and the vector $\mathbf{f} \in \mathbb{R}^m$ with elements

$$A_{ij} = \int_0^L k\, \varphi_i'(x)\, \varphi_j'(x)\, dx, \qquad f_i = \int_0^L f(x)\, \varphi_i(x)\, dx$$

we can also write the Galerkin conditions more conveniently as the linear system

$$\mathbf{A}\mathbf{c} = \mathbf{f}, \tag{26}$$

which one needs to solve for $\mathbf{c}$.

Example: Sine basis

Let us consider the Dirichlet heat equation

$$\begin{cases} -\dfrac{\partial^2 u}{\partial x^2}(x) = x & x \in (0, \pi) \\ u(0) = u(L) = 0, \end{cases}$$

i.e. where $k = 1$, $L = \pi$ and $f(x) = x$.

For the basis functions φ_j we select the sine basis

$$\varphi_j(x) = \sin(j\, x) \qquad \text{for } j = 1, \dots, m.$$

It is easy to see that all of them satisfy $\varphi_j(0) = \varphi_j(2\pi) = 0$. With these basis functions we can obtain the entries of $\mathbf{A}$ and $\mathbf{f}$ in equation (26) as

$$\begin{aligned} A_{ij} &= \int_0^\pi \frac{d}{dx}\big[\sin(i\, x)\big]\, \frac{d}{dx}\big[\sin(j\, x)\big]\, dx \\ &= i\, j \int_0^\pi \cos(i\, x) \cos(j\, x)\, dx = \frac{\pi\, i^2}{2}\, \delta_{ij} = \begin{cases} \frac{\pi\, i^2}{2} & i = j \\ 0 & i \neq j \end{cases} \end{aligned}$$

$$f_i = \int_0^\pi x \sin(i\, x)\, dx = \frac{(-1)^{i+1}\, \pi}{i}$$

For example for $m = 5$ we find the matrix and vector

$$\mathbf{A} = \frac{\pi}{2}\begin{pmatrix} 1 & & & & \\ & 4 & & & \\ & & 9 & & \\ & & & 16 & \\ & & & & 25 \end{pmatrix} \qquad \mathbf{f} = \pi\begin{pmatrix} 1 \\ -\frac{1}{2} \\ \frac{1}{3} \\ -\frac{1}{4} \\ \frac{1}{5} \end{pmatrix}.$$

Since $\mathbf{A}$ is diagonal we can directly compute the coefficient vector $\mathbf{c}$ as

$$\mathbf{c} = \begin{pmatrix} f_1/A_{11} \\ f_2/A_{22} \\ f_3/A_{33} \\ f_4/A_{44} \\ f_5/A_{55} \end{pmatrix} = \begin{pmatrix} 2 \\ -\frac{2}{8} \\ \frac{2}{27} \\ -\frac{2}{64} \\ \frac{2}{125} \end{pmatrix}$$

and the approximate solution becomes

$$u(x) = 2\sin(x) - \frac{2}{8}\sin(2x) + \frac{2}{27}\sin(3x) - \frac{2}{64}\sin(4x) + \frac{2}{125}\sin(5x).$$

Generalising to arbitrary m we find the solution coefficients

$$c_i = \frac{f_i}{A_{ii}} = \frac{2\,(-1)^{i+1}}{i^3} \qquad \forall\, i = 1, \ldots, m$$

leading to the solution function

$$u(x) = \sum_{i=1}^{m} \frac{2\,(-1)^{i+1}}{i^3}\sin(i\,x), \tag{27}$$

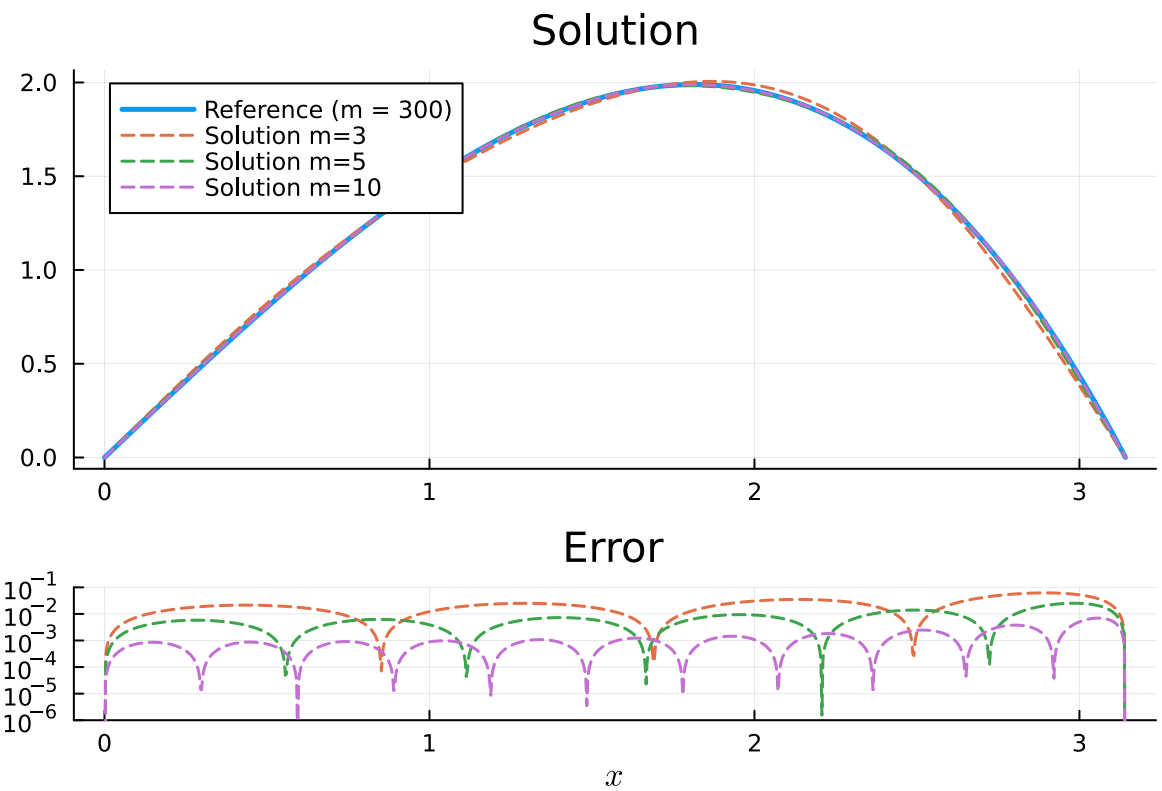
which can be implemented as:

```
sine_solution (generic function with 1 method)
```

```julia
function sine_solution(m, x)
    # Compute the function value of the numerical solution
    # for given m and x by evaluating (27)

    result = 0.0
    for i in 1:m
        result += 2 * (-1)^(i+1) / i^3 * sin(i * x)
    end
    result
end
```

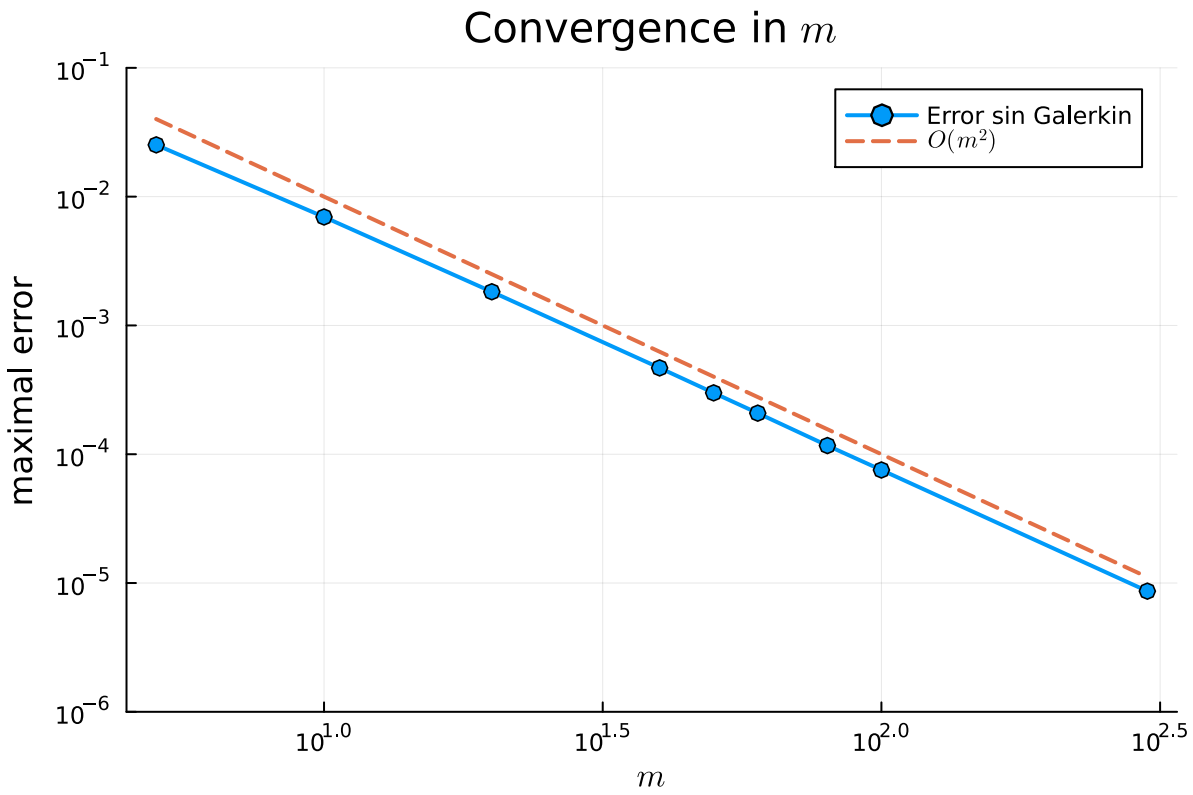
Choosing different values for m we obtain:

- Show reference: ☑
- Show $m = 10$: ☑



We see that already for 5 $\sin$ basis functions it becomes visually very hard to see the difference to the reference solution with 300 basis functions.

Numerically we observe a **quadratic convergence** in a log-log plot:




```julia
let
    x = range(0, π; length=1000)
    reference(x) = sine_solution(800, x)

    ms = [5, 10, 20, 40, 50, 60, 80, 100, 300]
    errors = Float64[]
    for m in ms
        error = [abs(sine_solution(m, xᵢ) - reference(xᵢ)) for xᵢ in x]
        push!(errors, maximum(error))
    end

    p = plot(ms, errors, yaxis=:log, xaxis=:log, lw=2, mark=:o, title=L"Convergence in $m$", xlabel=L"m", ylabel="maximal error", label="Error sin Galerkin")
    plot!(p, ms, 1 ./ms.^2, ls=:dash, lw=2, label=L"O(m^2)")

    ylims!(p, (1e-6, 1e-1))
    xticks!(p, 10.0 .^ (0:0.5:3))
    yticks!(p, 10.0 .^ (0:-1:-6))

    p
end
```

Comparison and conclusion

⚠ TODO ⚠

Some conclusion chapter to provide direct comparison between finite differences and Galerkin methods and from that some conclusion why Galerkin methods can be better (numerical stability, less unknowns for a targeted accuracy etc.)

```
TODO("Some conclusion chapter to provide direct comparison between finite differences and Galerkin methods and from that some conclusion why Galerkin methods can be better (numerical stability, less unknowns for a targeted accuracy etc.)")
```

⚠ TODO ⚠

To introduce FEM we need the concept of a mass matrix in the Galerkin development above as the basis functions are not orthonormal. See chapter 10.5 of Driscoll Brown

```
TODO("To introduce FEM we need the concept of a mass matrix in the Galerkin development above as the basis functions are not orthonormal.

    See chapter 10.5 of Driscoll Brown
    ")
```

Optional: Finite elements

A widely employed set of basis functions for Galerkin approximations are the hat functions $\varphi_i = H_i$, which we already discussed in the chapter on Interpolation (chapter 7). Recall, that given a set of nodes $x_0 < x_1 < \cdots < x_n$ the hat functions are defined as

$$H_i(x) = \begin{cases} \frac{x - x_{i-1}}{x_i - x_{i-1}} & \text{if } i > 0 \text{ and } x \in [x_{i-1}, x_i] \\ \frac{x_{i+1} - x}{x_{i+1} - x_i} & \text{if } i < n \text{ and } x \in [x_i, x_{i+1}] \\ 0 & \text{otherwise} \end{cases}$$

for $i = 0, \ldots, n$. In this discussion we will again only consider equispaced nodes for simplicity, i.e. we take the case with $0 = x_0 < x_1 < \cdots < x_n = L$ and where $x_{i+1} - x_i = \frac{L}{n}$. Defining $h = \frac{L}{n}$ the hat functions can thus equally be expressed as

$$H_i(x) = \begin{cases} \frac{x - x_{i-1}}{h} & \text{if } i > 0 \text{ and } x \in [x_{i-1}, x_i] \\ \frac{x_{i+1} - x}{h} & \text{if } i < n \text{ and } x \in [x_i, x_{i+1}] \\ 0 & \text{otherwise} \end{cases} \tag{28}$$

for $i = 0, \ldots, n$.

Due to the cardinality property of the hat functions $\varphi_i = H_i(x)$, i.e. $H_i(x_j) = \delta_{ij}$, the expansion (24) of the unknown solution function can be simplified to

$$u(x) = \sum_{j=1}^{m} c_j \varphi_j(x) = \sum_{i=1}^{n-1} u_i \, H_i(x), \tag{29}$$

where $u_i = u(x_i)$, i.e. the numerical solution at the nodal points. Note that the last sum in (29) is deliberately truncated to omit the hat functions H_0 and H_n. The nodal points of these two functions are $x_0 = 0$ and $x_n = L$, respectively, where these functions take by definition the value 1. As a result these functions are unable to satisfy the condition $0 = \varphi_j(0) = \varphi_j(L)$ underlying V_0 and are thus excluded.

The importance of the hat functions for Galerkin methods stems from the fact that each hat function $H_i(x)$ is only non-zero in the interval $[x_{i-1}, x_{i+1}]$. As a result when evaluating the Galerkin conditions (26) one never has to integrate over the entire domain $[0, L]$, but only a much smaller subset of this interval where the involved hat functions are non-zero — a noteworthy reduction of the computational effort.

As an example consider the evaluation of the elements of the matrix $\mathbf{A} \in \mathbb{R}^{m \times m} = \mathbb{R}^{(n-1)\times(n-1)}$. First note that the derivatives H_i' are non-zero only where H_i itself is non-zero, i.e. $[x_{i-1}, x_{i+1}]$. Therefore the product $H_i'\, H_j'$ is only non-zero in the intersection of $[x_{i-1}, x_{i+1}]$ and $[x_{j-1}, x_{j+1}]$.

This intersection is empty, thus the matrix element A_{ij} zero, whenever $|i - j| > 2$, i.e. when the two indices are more than two apart from each other. For $|i - j| = 2$ the intersection contains only a single point, which similarly implies $A_{ij} = 0$. As a consequence

$$A_{i,i+d} = \int_0^L k\, H_i'(x)\, H_{i+d}'(x)\, dx = 0 \qquad \text{if } d \geq 2.$$

where $i = 1, \ldots, n-3$ and similarly by symmetry

$$A_{i+d,i} = \int_0^L k\, H_{i+d}'(x)\, H_i'(x)\, dx = 0 \qquad \text{if } d \geq 2.$$

again for $i = 1, \ldots, n-3$

Now we consider the remaining three cases, namely

$$
\begin{aligned}
A_{ii} &= \int_0^L k\, H_i'(x)\, H_i'(x)\, dx \\
&= \int_{x_{i-1}}^{x_i} k\, H_i'(x)\, H_i'(x)\, dx + \int_{x_i}^{x_{i+1}} k\, H_i'(x)\, H_i'(x)\, dx \\
&= \int_{x_{i-1}}^{x_i} k \cdot \frac{1}{h} \cdot \frac{1}{h}\, dx + \int_{x_i}^{x_{i+1}} k \cdot \frac{-1}{h} \cdot \frac{-1}{h}\, dx \\
&= \frac{k}{h^2} \left[\int_{x_{i-1}}^{x_i} 1\, dx + \int_{x_i}^{x_{i+1}} 1\, dx \right] \\
&= \frac{2k}{h}
\end{aligned}
$$

for $i = 1, \ldots, n-1$ as well as

$$
\begin{aligned}
A_{i+1,i} = A_{i,i+1} &= \int_0^L k\, H_i'(x)\, H_{i+1}'(x)\, dx \\
&= \int_{x_i}^{x_{i+1}} k\, H_i'(x)\, H_{i+1}'(x)\, dx \\
&= \int_{x_i}^{x_{i+1}} k \cdot \frac{-1}{h} \cdot \frac{1}{h}\, dx \\
&= -\frac{k}{h}.
\end{aligned}
$$

for $i = 1, \ldots, n-2$. We recover a tridiagonal matrix

$$
\mathbf{A} = \frac{k}{h} \begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & -1 & 2 & \ddots & \\ & & \ddots & \ddots & -1 \\ & & & -1 & 2 \end{pmatrix} \in \mathbb{R}^{(n-1)\times(n-1)} \tag{30}
$$

⚠ TODO ⚠

Properly work the mass matrix stuff into this.

```
TODO("Properly work the mass matrix stuff into this.")
```

Mass matrix

$$
\begin{aligned}
M_{ii} &= \int_0^L H_i(x)\, H_i(x)\, dx \\
&= \int_{x_{i-1}}^{x_i} H_i(x)\, H_i(x)\, dx + \int_{x_i}^{x_{i+1}} H_i(x)\, H_i(x)\, dx \\
&= \int_{x_{i-1}}^{x_i} \frac{(x - x_{i-1})(x - x_{i-1})}{h^2}\, dx + \int_{x_i}^{x_{i+1}} \frac{(x_{i+1} - x)(x_{i+1} - x)}{h^2}\, dx \\
&= \frac{(x_i - x_{i-1})^3}{3h^2} + \frac{(x_{i+1} - x_i)^3}{3h^2} \\
&= \frac{h^3}{3h^2} + \frac{h^3}{3h^2} \\
&= \frac{2}{3} h
\end{aligned}
$$

and

$$
\begin{aligned}
M_{i,i+1} &= \int_0^L H_i(x)\, H_{i+1}(x)\, dx = \int_{x_i}^{x_{i+1}} H_i(x)\, H_{i+1}(x)\, dx \\
&= \int_{x_i}^{x_{i+1}} \frac{(x_{i+1} - x)(x - x_{i+1})}{h^2}\, dx \\
&= -\frac{(x_{i+1} - x_i)^3}{3h^2} \\
&= -\frac{h}{3}
\end{aligned}
$$

Finally we consider the elements of the vector $\mathbf{f}$. Since again H_i is only non-zero in $[i-1, i+1]$, we need to perform the two integrals

$$f_i = \int_0^L f(x)\, H_i(x)\, dx = \int_{x_{i-1}}^{x_i} f(x)\, H_i(x)\, dx + \int_{x_i}^{x_{i+1}} f(x)\, H_i(x)\, dx$$

for $i = 1, \ldots, n-1$. One can show that to a very good approximation one can replace $f(x)$ by the average value of the function over the respective integrals, i.e.

$$\begin{aligned}
\int_{x_{i-1}}^{x_i} f(x)\, H_i(x)\, dx &\approx \frac{f(x_{i-1}) + f(x_i)}{2} \int_{x_{i-1}}^{x_i} H_i(x)\, dx = \frac{f(x_{i-1}) + f(x_i)}{2} \cdot \frac{h}{2} \\
\int_{x_i}^{x_{i+1}} f(x)\, H_i(x)\, dx &\approx \frac{f(x_i) + f(x_{i+1})}{2} \int_{x_i}^{x_{i+1}} H_i(x)\, dx = \frac{f(x_i) + f(x_{i+1})}{2} \cdot \frac{h}{2}.
\end{aligned}$$

In particular this approximation converges quadratically as $h \to 0$, which turns out to be exactly the order of approximation of the finite element method itself. Therefore putting in additional efforts to compute these integrals more accurately would not even improve the accuracy of our final result.

Putting these developments tothere we obtain for $i = 1, \ldots, n-1$ that

$$\begin{aligned}
f_i &= \frac{f(x_i) + f(x_{i-1})}{2} \frac{h}{2} + \frac{f(x_{i+1}) + f(x_i)}{2} \frac{h}{2} \\
&= \frac{h}{4} \Big(f(x_{i-1}) + 2f(x_i) + f(x_{i+1}) \Big).
\end{aligned} \tag{31}$$

By solving the linear system

$$(\mathbf{A} + \mathbf{M})\, \mathbf{u} = \mathbf{f}$$

we then obtain the coefficients $\mathbf{u} = (u_1, u_2, \ldots, u_{n-1})^T$ in the expansion (29). Notably, due to the cardinality property of the hat functions $u_i = u(x_i)$, i.e. these coefficients are equal to evaluating our numerical approximation $u(x)$ at the nodal points $\{x_i\}_{i=1}^{n-1}$.

An implementation of this approch is given below:

```
heat_equation_1d_fem (generic function with 1 method)
```

```julia
function heat_equation_1d_fem(f, k, L, n)
    # f:  Function describing the external heat source
    # k:  thermal conductivity
    # L:  Length of the metal rod
    # N:  Number of nodal points for the finite element scheme

    h = L / n                   # Step size
    x = [i * h for i in 0:n]    # Nodal points (x₀, x₁, ..., xₙ)
    x_inner = x[2:n]            # Inner nodal points (x₁, ..., xₙ₋₁)


    # Build A and f: Note that these are a (n-1) × (n-1) matrix
    # respectively a vector of length (n-1)
    A = k/h * SymTridiagonal(2ones(n-1), -ones(n-2))
    M = h/3 * SymTridiagonal(2ones(n-1), -ones(n-2))
    f = h/4 * [f(x[i-1]) + 2f(x[i]) + f(x[i+1])
               for i in 2:n]   # Skip first/last nodal point, i.e. x[1] = x₀

    u = (A + M) \ f
    (; x=x_inner, u)
end
```

Let us return to the example we considered with the sine basis, i.e.

$$\begin{cases} -\dfrac{\partial^2 u}{\partial x^2}(x) = x & x \in (0, \pi) \\ u(0) = u(L) = 0, \end{cases}$$

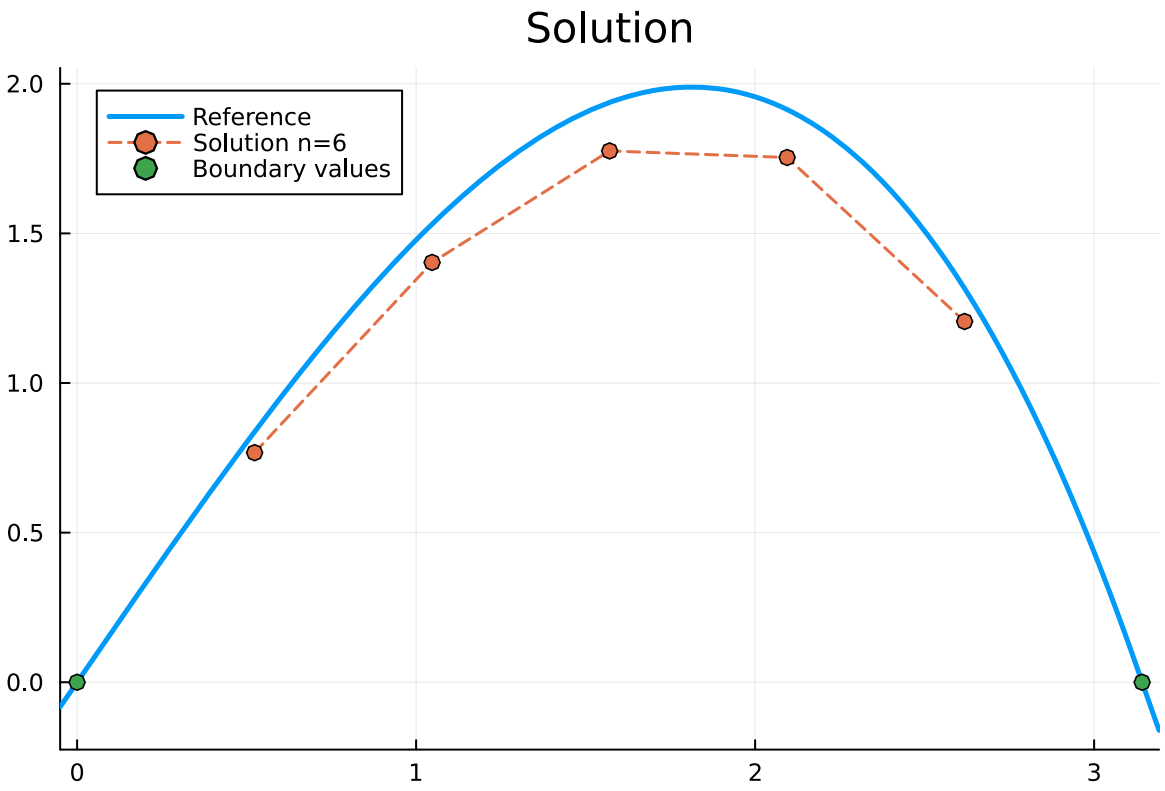
i.e. the Dirichlet heat equation (20) with $k = 1$, $L = \pi$ and $f(x) = x$.

As a reference solution we consider the solution using the sine basis with $m = 300$, which we know to be very accurate for this problem:

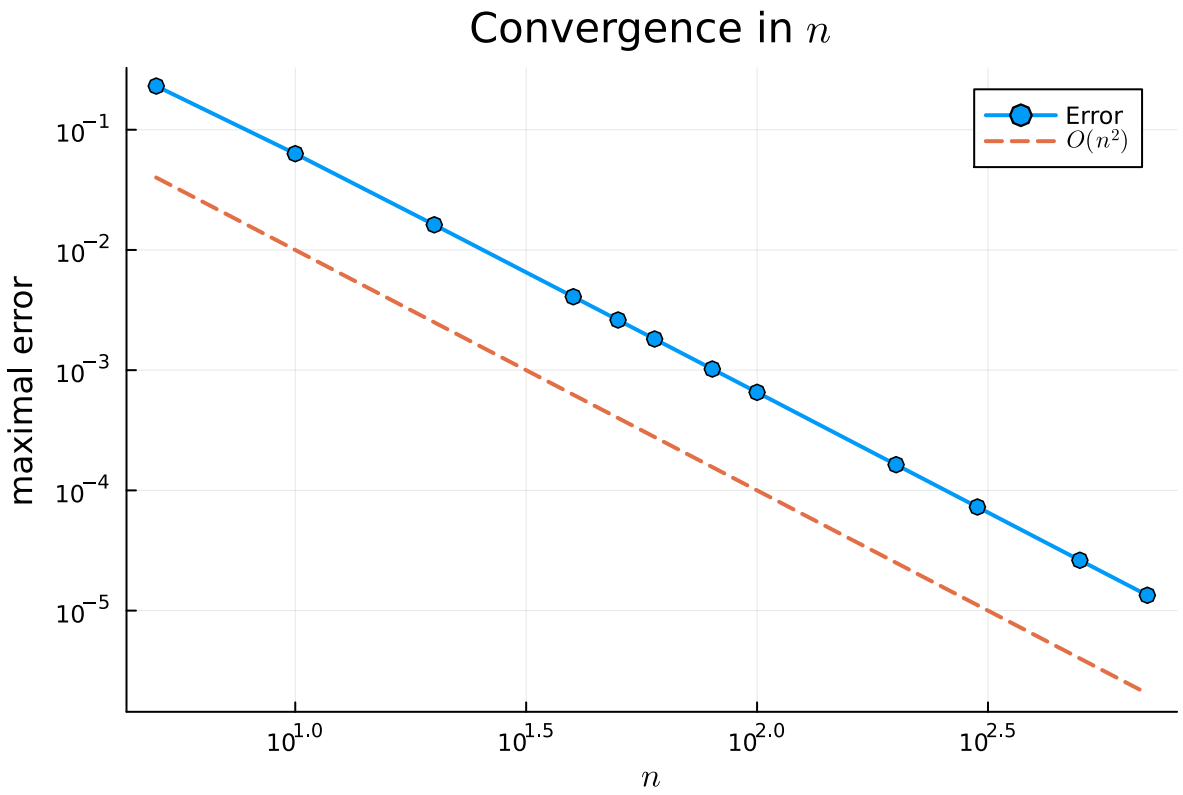
```
reference (generic function with 1 method)
```

```julia
reference(x) = sine_solution(300, x)
```

- n = 6




```
let
    # Parameters of the problem
    f(x) = x
    k = 1
    L = π

    # Plot the reference
    p = plot(reference; lw=2.5, label="Reference", title="Solution", xlims=(-0.05,
    L + 0.05))

    # Compute FEM solution and plot it
    result = heat_equation_1d_fem(f, k, L, n)
    plot!(p, result.x, result.u; lw=1.5, ls=:dash, label="Solution n=$n", mark=:o)

    scatter!([0, L], [0, 0], label="Boundary values", mark=:o, c=3)
end
```




```julia
let
    f(x) = x
    k = 1
    L = π

    ns = [5, 10, 20, 40, 50, 60, 80, 100, 200, 300, 500, 700]
    errors = Float64[]
    for n in ns
        result = heat_equation_1d_fem(f, k, L, n)
        error = abs.(result.u - reference.(result.x))
        push!(errors, maximum(error))
    end

    p = plot(ns, errors, yaxis=:log, xaxis=:log, lw=2, mark=:o, title=L"Convergence in $n$", xlabel=L"n", ylabel="maximal error", label="Error")
    plot!(p, ns, 1 ./ns.^2, ls=:dash, lw=2, label=L"O(n^2)")

    xticks!(p, 10.0 .^ (0:0.5:3))
    yticks!(p, 10.0 .^ (0:-1:-6))
end
```

The finite element method is one of the most widely employed approaches in science and engineering to solve differential equations. With this short instroduction we only scratched the surface.

More information on the approach can be found for example in chapter 10.6 of Driscoll, Brown: *Fundamentals of Numerical Computation*.

Show outline ☑

Numerical analysis

1. Introduction
2. The Julia programming language
3. Revision and preliminaries
4. Root finding and fixed-point problems
5. Direct methods for linear systems
6. Iterative methods for linear systems
7. Interpolation
8. Numerical integration
9. Numerical differentiation
10. Boundary value problems
11. Eigenvalue problems
12. Initial value problems